Welcome to Jenkins!

Jenkins Installation and Pipeline Documentation

Table of Contents:

- **Introduction** 3
- **Module 1:** Installation of Jenkins 5
- **Module 2**: Creating Pipeline Job in Jenkins.....11
- **Module 3**: Adding Webhook trigger to Jenkins.....19

NAVEEN

INTRODUCTION

Jenkins is an open-source automation server widely used for continuous integration and continuous delivery (CI/CD). It allows development teams to automate the building, testing, and deployment of applications, thereby improving software quality, reducing manual errors, and accelerating release cycles.

This document provides a comprehensive guide to installing Jenkins, setting up pipelines, and implementing automated deployment workflows. It is designed to assist developers, DevOps engineers, and system administrators in understanding and configuring Jenkins for enterprise-grade applications.

Key objectives of this documentation include:

1. **Installation & Configuration**: Step-by-step instructions for installing Jenkins on supported operating systems, setting up plugins, and configuring initial security settings.

2. **Pipeline Setup**: Detailed examples of Declarative pipelines, including build, test, deployment stages, and post-build notifications.

3. **Best Practices**: Guidance on structuring Jenkins jobs, managing credentials securely, and optimizing pipelines for reliability and maintainability.

4. **Automation with Notifications**: Integration with notification systems such as AWS SNS, email, and Slack to ensure stakeholders are informed of build and deployment statuses.

By following this documentation, teams can streamline their development workflows, ensure consistent deployments across environments, and leverage Jenkins' powerful automation capabilities to achieve faster and more reliable software delivery.

PRE-REQUISITES

- Aws account with Sign Credentials.
- Permission to EC2 Full access.
- Storage Configuration of minimum 15 GB
- RAM minimum 1 GB.
- AWS CLI (Command Line Interface) installed for certain tasks.
- GIT Repo Access
- **Java**: Java 11 or Java 17 (required to run Jenkins)
- SSM role with permission attached

Module 1

Jenkins Installation on AWS EC2 (Production Setup Guide):

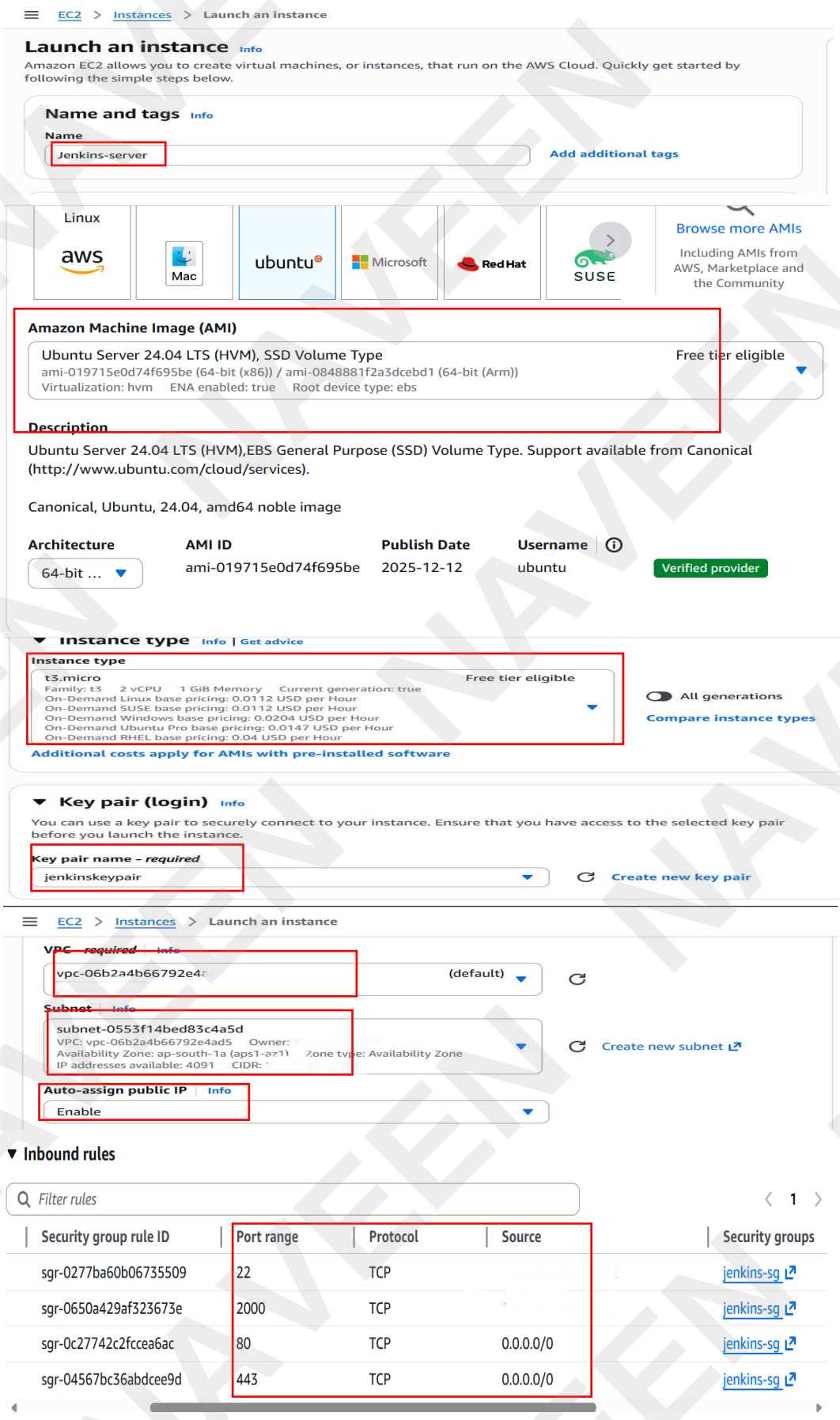
1.1 Overview

This section describes the complete process of installing Jenkins on an AWS EC2 instance, including:

- Launching an EC2 instance
- Configuring security groups
- Installing Java
- Installing Jenkins
- Configuring firewall rules
- Accessing Jenkins via browser

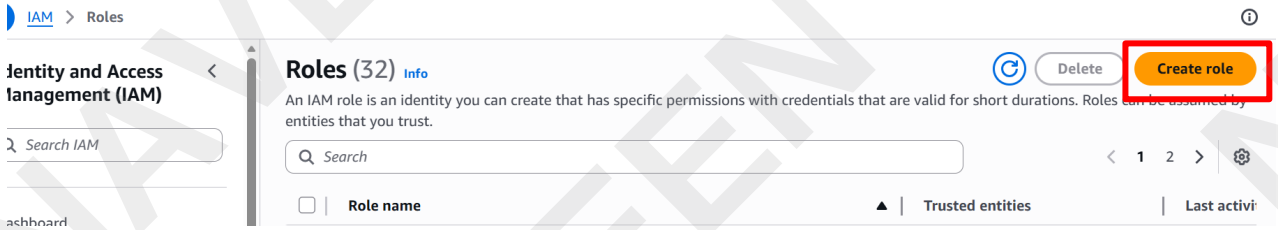
1.2 Launch an EC2 Instance

1. Go to: https://aws.amazon.com
2. Sign in to the AWS Management Console
3. Navigate to EC2 Dashboard
4. Click Launch Instance:
   1) Enter name of the sever
   2) Select Ubuntu 24.04 as a standard version
   3) Select Instance type (use t3. micro as a beginning with 1 gb memory and 2 vCPU)
   4) Create a new key pair
   5) Select the VPC and subnet
   6) Enable the auto assign IP
   7) Configure storage of minimum 15 gb of gp3 required.
   8) Create security group allow inbound:
      Port 22 from your IP
      Port 8080 from your IP or whosoever IP required
      Port 80 and 443 from 0.0.0.0/0

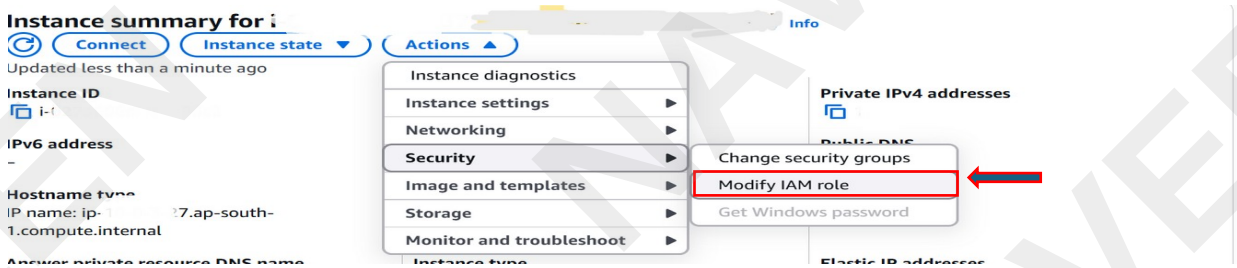


1.3 Attach SSM role to EC2 Instance

1. 1. Go to IAM > Roles >create new role (if Role is not created)

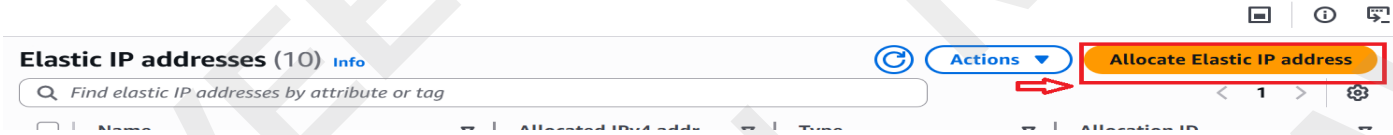


2. Select AWS service and EC2 in use case
3. Attach policy
4. AmazonSSMFullAccess
5. AmazonSSMManagedInstanceCore
6. AmazonSSMRunCommandAccess
7. ListCommandInvocations
8. Now Come to instance Page and Select Actions > Security > Modify IAM Role
9. Attach the role created



1.4 Attach Elastic IP to EC2

Select Allocate new Elastic IP:



Select IP and associate it with the instance (Jenkins-server).

Elastic IPs provide **stability, reliability, and control** over your instance's public IP, making them ideal for production workloads, failover setups, and predictable network access.

1.4 Connect to EC2

Connect to EC2 Through SSH into it using terminal / PowerShell /ubuntu.

ssh -i your-key.pem ubuntu@<public-ip>

OR

Through SSM session Manager, if SSM agent is installed.

1.5 Installing Jenkins in ubuntu

Open Jenkins documentation file in browser: Linux

Install Java: `sudo apt update (#Update APT packages in Ubuntu)`
`sudo apt install fontconfig openjdk-21-jre (install Java JDK )`
`java –version (verify java installation and version installed)`

If the installation was successful, you should see an output like the following:

```
openjdk 21.0.8 2025-07-15

OpenJDK Runtime Environment (build 21.0.8+9-Debian-1)

OpenJDK 64-Bit Server VM (build 21.0.8+9-Debian-1, mixed mode, sharing)
```

Install LTS: A LTS (Long-Term Support) release is chosen every 12 weeks from the stream of regular releases as the stable release for that time

```
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key


echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null
```

Install Jenkins:

```
sudo apt install jenkins
```

Start and Enable Jenkins:

```
sudo systemctl start jenkins

sudo systemctl enable jenkins

sudo systemctl status jenkins

You should see:

Active: active (running)
```

Allow Firewall (If UFW Enabled)

```
sudo ufw allow 8080

sudo ufw allow OpenSSH

sudo ufw enable
```

Browse Jenkins through Public IP of EC2 with Port number:

http://13.232.xxx.xxx:8080

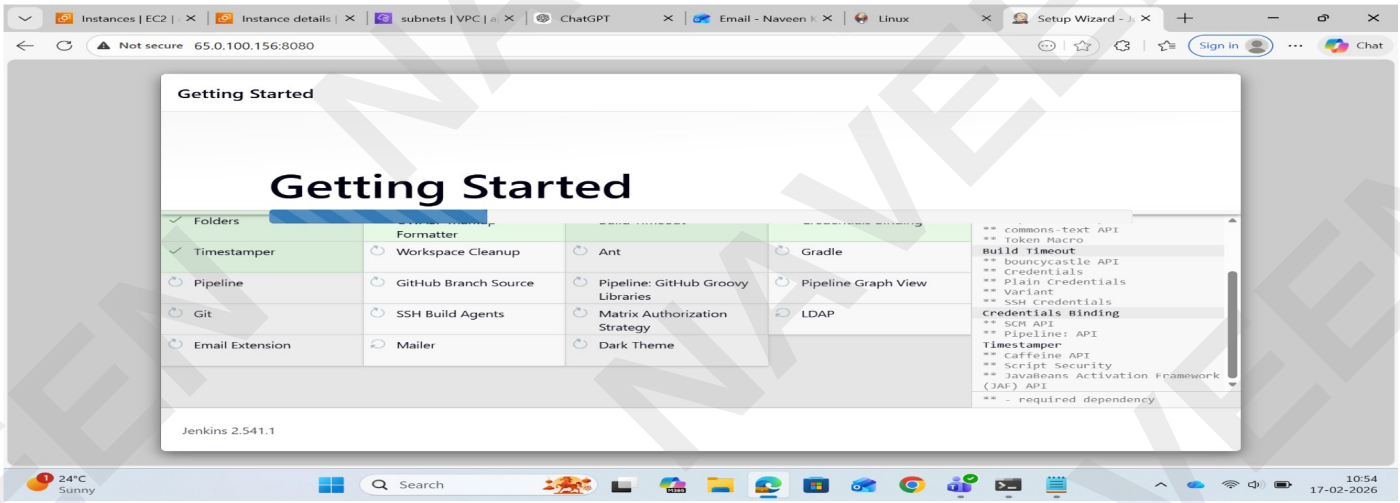
Unlock Jenkins

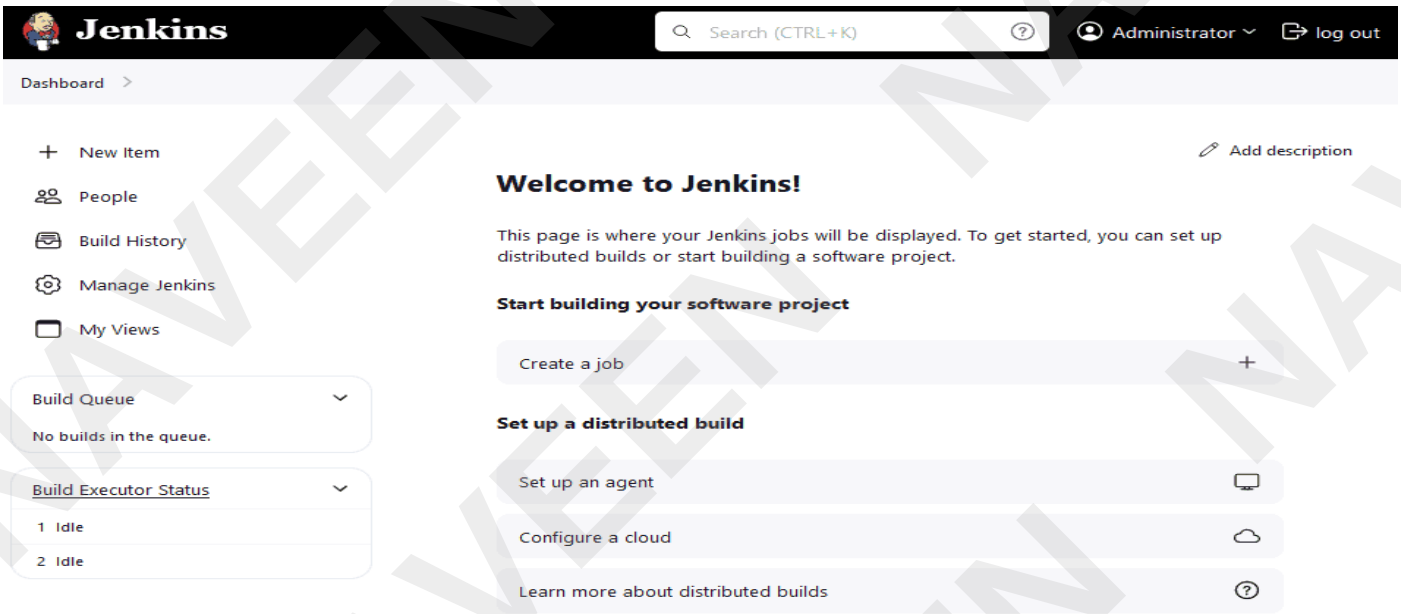
Jenkins is opened with Admin page first, need to unlock the Jenkins with password.

Password will be found in **sudo cat /var/lib/jenkins/secrets/initialAdminPassword**

Copy the password and paste it on UI console.

Install the plugins required.





Create Admin User

Enter:

- Username
- Password
- Full Name
- Email

Click **Save and Continue**

Post-Installation Recommendations

- Set up Nginx reverse proxy
- Add domain name to IP
- Enable HTTPS using SSL certificate
- Install required plugins (Git, Pipeline, Docker, AWS, etc.)
- Set up automatic backups
- Configure SNS or Email notifications

Module 2

Creating Your First Jenkins Pipeline

2.1 Overview

A **Jenkins Pipeline** is a set of automated processes that define how your application is built, tested, and deployed. Pipelines can be **Declarative** (simpler, structured) or **Scripted** (more flexible).

In this section, we will create a **Declarative Pipeline** for a sample project, demonstrating:

- Source code checkout from GitHub
- Build stage
- Deployment stage
- Notifications on success/failure

2.2 Prerequisites

Before creating a pipeline:

1. Jenkins is installed and running.
2. Git is installed on the Jenkins server.
3. You have **repository access** (GitHub/Bitbucket/GitLab).
4. Necessary plugins are installed:
    a. **Pipeline**
    b. **Git**
    c. **AWS CLI** (for AWS deployments)
    d. **Email Extension** or **AWS SNS** (for notifications)

2.3 Create a New Pipeline

1.Click on new item in the left side of the main page:

2. Enter the Pipeline name you want and select Pipeline in the below options

3. Under **Configure** We have

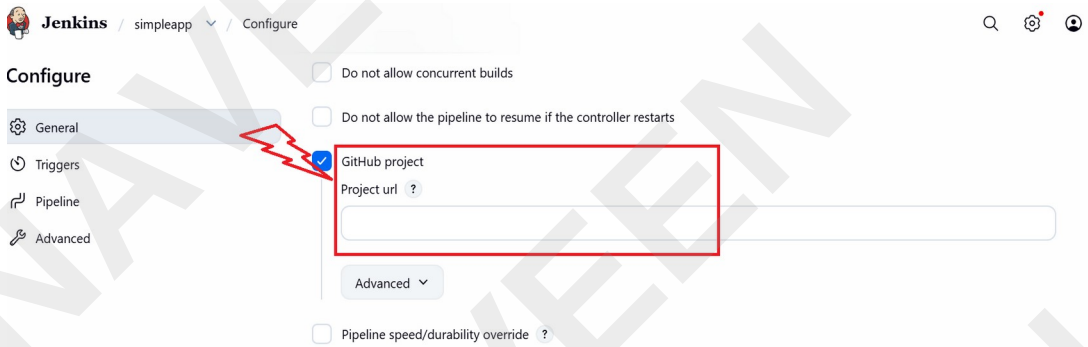
1. **General**
2. **Trigger**
3. **Pipeline**
4. **Advance**

2.4 Configure the Pipeline

2.4.1. General

Under general Section

1. Provide the description whatever is suitable for the pipeline
2. Check GitHub Project and provide the HTTPS address of the repo



3. Select Discard old builds if you want to delete them after build and mention the number of day or number of builds as per your requirement.

4. Check on Do not allow Concurrent Builds, as this avoids the conflicts among different builds. We can also add, whether you abort Previous builds or current Builds.

5. **Optional:** We can add Parameters to pass, if you do not want to Hard code the values in Jenkins file along with file name and path.

2.4.2. Trigger

Triggers define how and when a Jenkins job starts automatically. They allow builds to run in response to events such as code changes, schedules, upstream jobs, or external requests.

Build after other projects are built

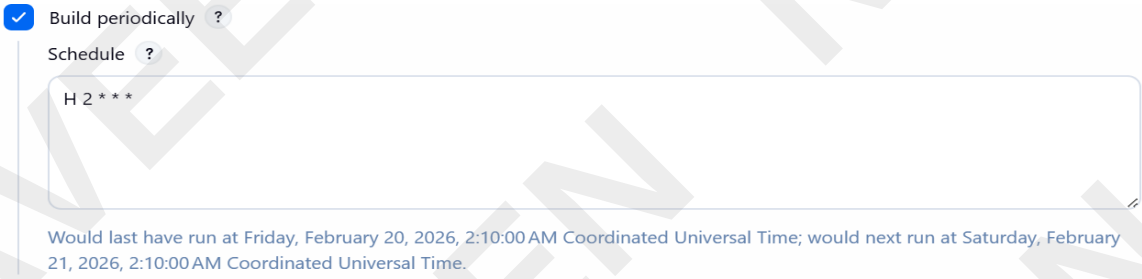
Starts this job after one or more specified Jenkins jobs complete.

- Typically used for **pipeline chaining**
- Can trigger on:
  - Success
  - Unstable
  - Failure (optional)
- Useful for multi-stage CI/CD flows (e.g., build → test → deploy)
- **Example:**
  Run deployment job after application build job finishes successfully.

Build periodically

Runs the job on a fixed schedule using **cron syntax**, regardless of code changes.

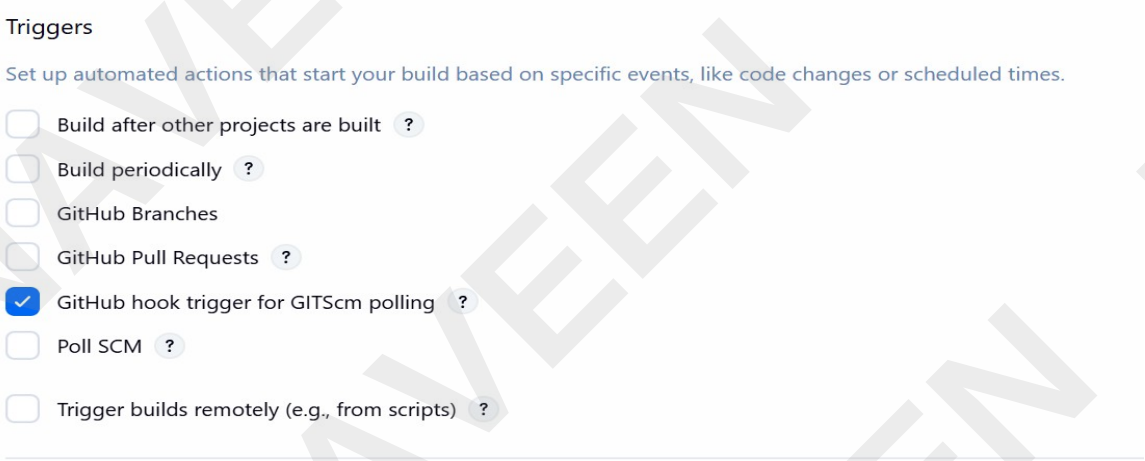
- Time-based automation
- No repository polling
- Best for maintenance or recurring tasks
- H 2 * * * == Runs once daily at ~2 AM.



GitHub hook trigger for GITScm polling

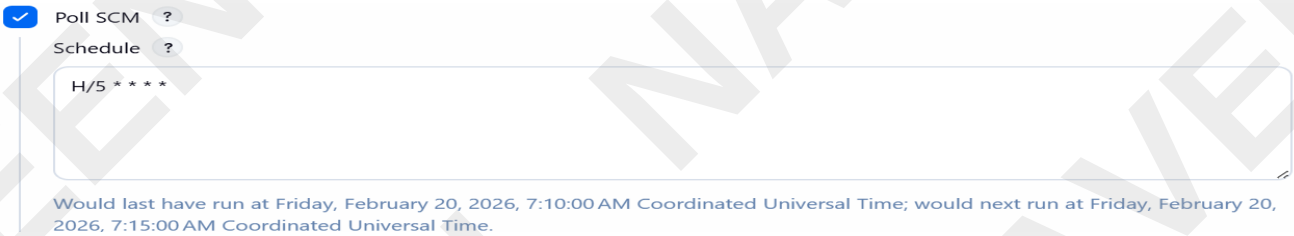
Starts a build when Jenkins receives a GitHub webhook event (push).

- Real-time trigger
- No polling delay
- Requires GitHub webhook configured to: **http(s)://<jenkins-url>/github-webhook/**



Poll SCM

Jenkins periodically checks the repository for changes using cron syntax. Works without Webhook. If changes are detected → build starts.



2.4.3. Pipeline Section

Definition

Specifies where the pipeline script (`Jenkinsfile`) is stored.

- **Pipeline script**

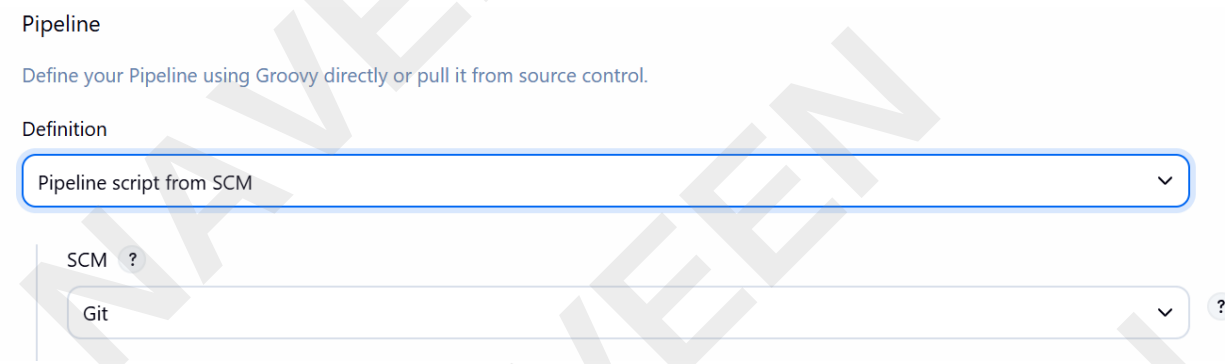
  The pipeline Groovy code is written directly inside the Jenkins job configuration.

- **Pipeline script from SCM (Recommended)**

  Loads the `Jenkinsfile` from a source control repository (e.g., Git).

SCM

Defines the source control system containing the Jenkinsfile.



Repositories

Specifies the repository containing the Jenkinsfile.

Repository URL: Git repository HTTPS or SSH URL.

Example: https://github.com/org/project.git

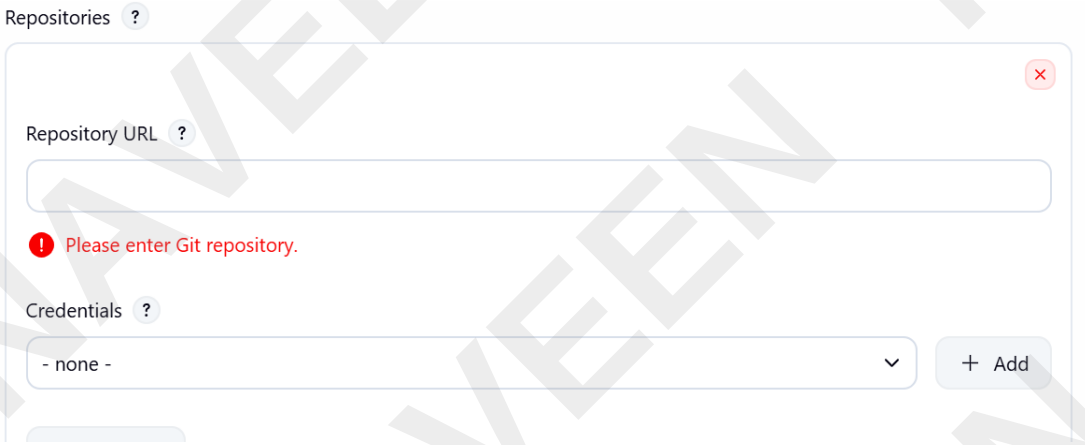
Credentials

Authentication used by Jenkins to access the repository.

Examples:

- GitHub PAT (Personal Access Token)
- SSH key
- Username/password

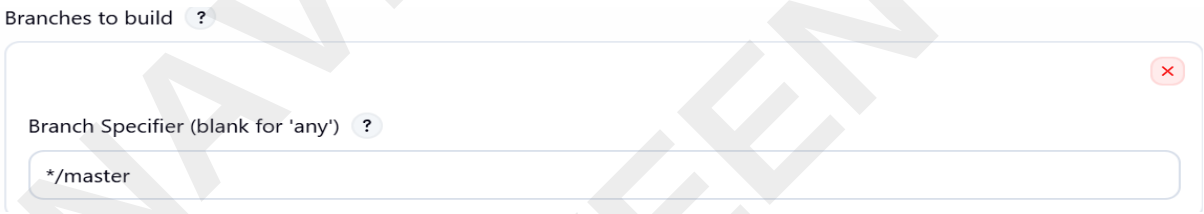
Requirement:
Must have at least read access to repository.



Branches to build

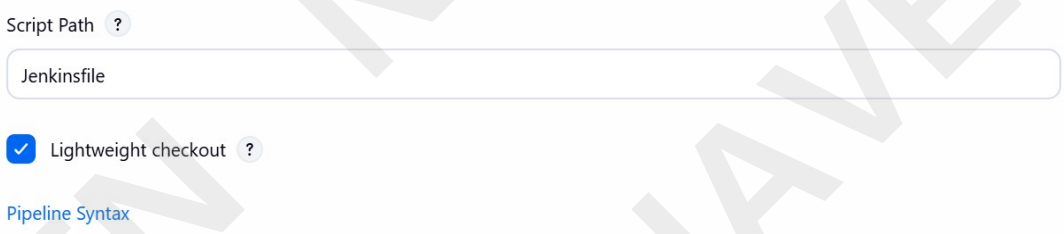
Specifies which branch Jenkins should fetch the Jenkinsfile from.

Branch Specifier: Git branch name with */ , Example Shown below



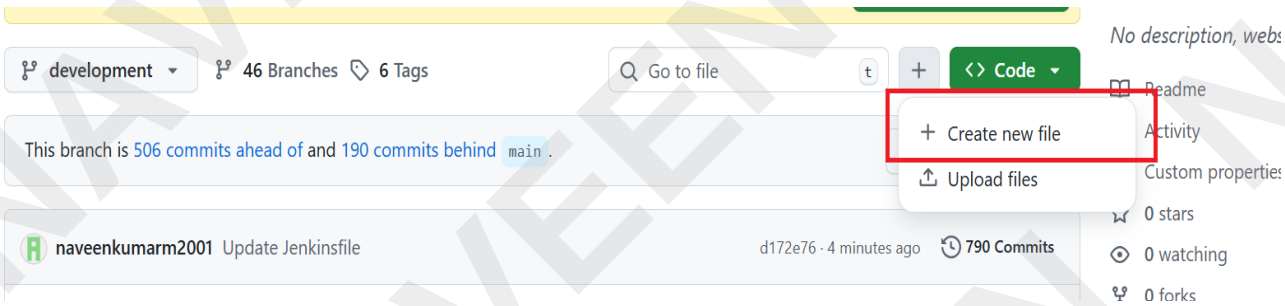
Script Path (Advanced)

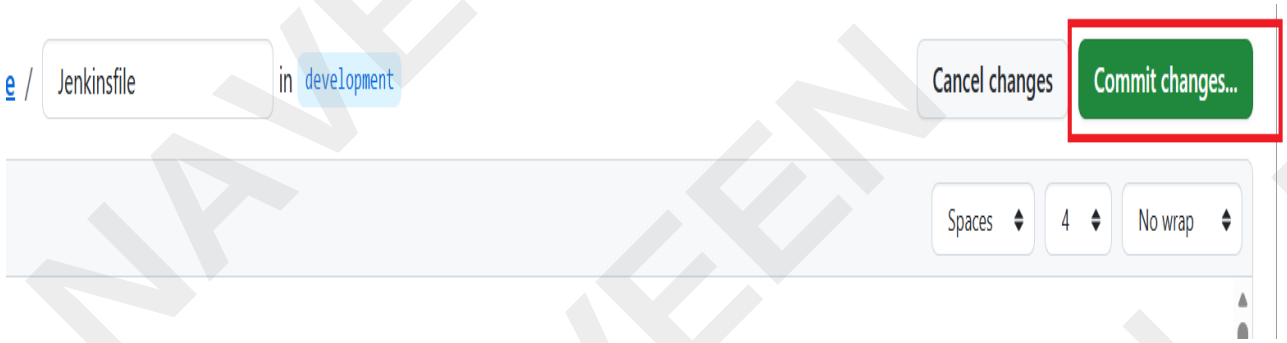
Location of Jenkinsfile inside repository.



2.4.4. Jenkinsfile

- Create a new file in Git Project repo with same branch you mentioned during Jenkins configuration.
- Enter Name as **Jenkinsfile** or the name you mentioned in the script path.
- Write the Pipeline script as required for the Job we need. We can also use VS code editor for indentation support and writing Script.
- Once the script is ready, Commit the script in the





Module 3

Adding Webhook Trigger to Jenkins

3.1 Overview

A **webhook trigger** enables Jenkins to automatically start a pipeline when events occur in a source control system such as GitHub, GitLab, or Bitbucket.
Instead of periodically polling the repository, Jenkins receives a real-time HTTP notification when code changes are pushed.

This provides faster builds, reduced server load, and immediate CI/CD feedback.

3.2 How Webhook Trigger Works

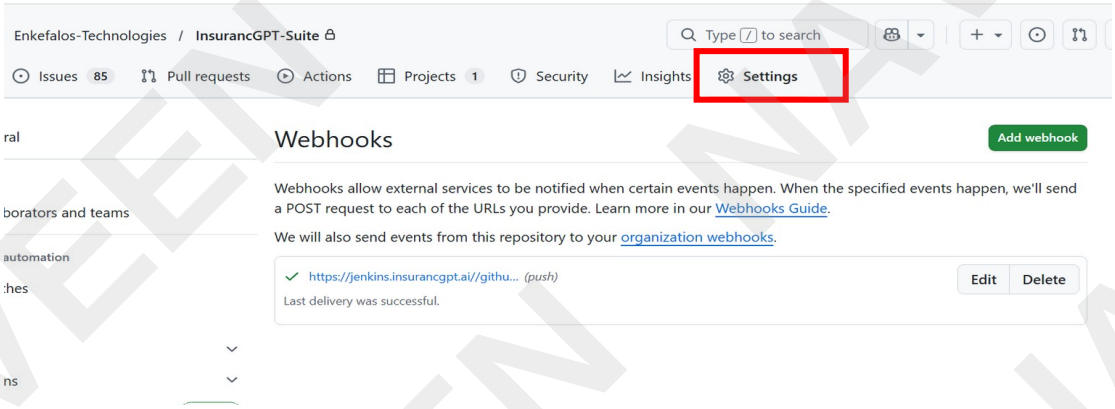
1. A developer pushes code to the repository.
2. The Git provider sends an HTTP POST request (webhook) to Jenkins.
3. Jenkins receives the event.
4. The configured job or pipeline starts automatically.

3.3 Benefits of Webhook Triggers

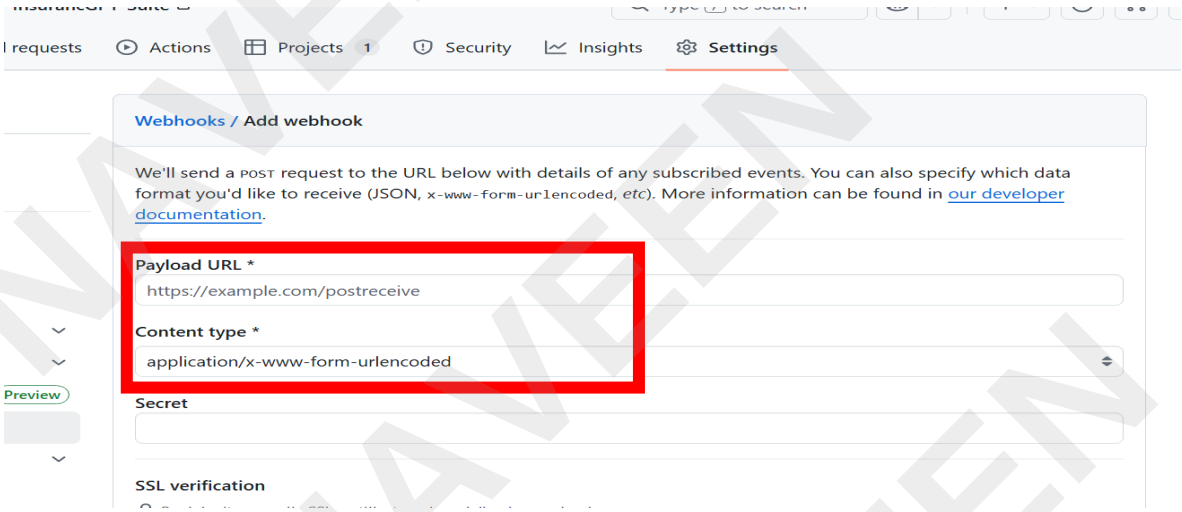
- Real-time build execution
- No polling delays
- Lower Jenkins resource usage
- Immediate CI/CD feedback
- Accurate event-based triggering

3.4 Repository Webhook Configuration

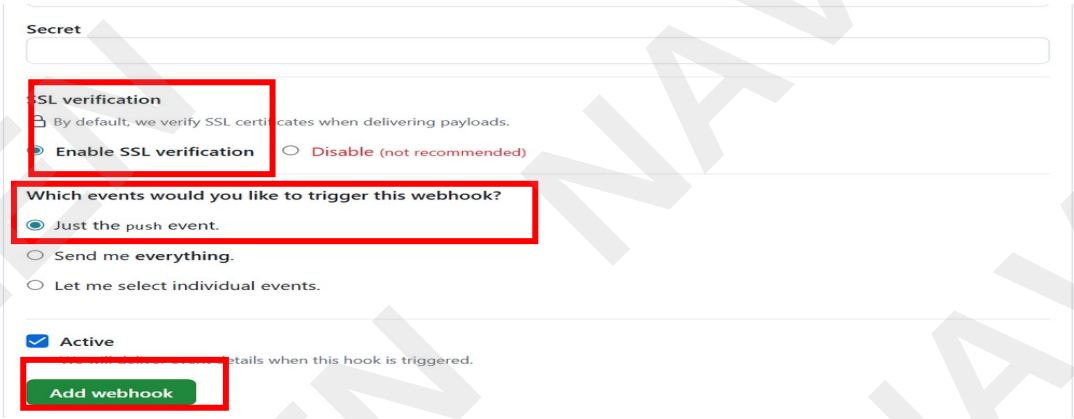
- Add a webhook in the repository settings pointing to Jenkins.
- Login to Github and move to your Repository of the project
- Move to Repo Settings



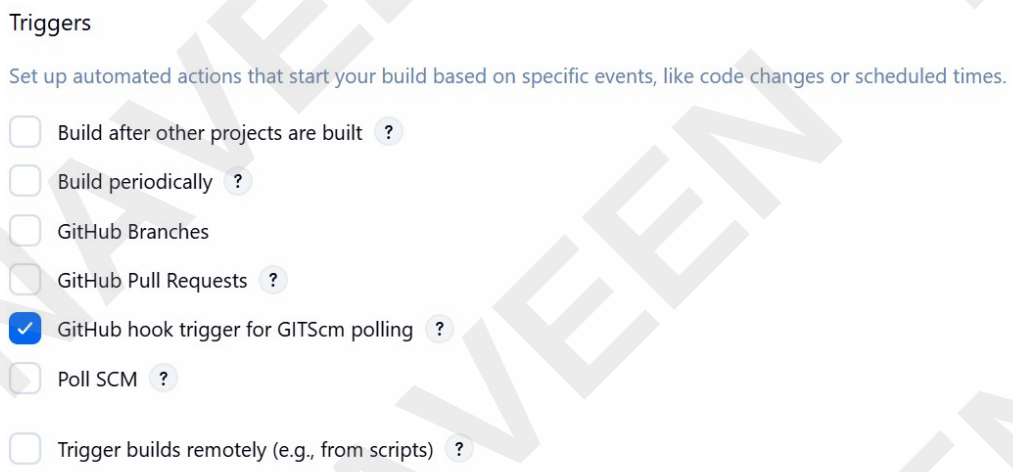
- Enter the payload URL: **http(s)://<jenkins-url>/github-webhook/**
- Enter Content type: application/x-www-form-urlencodd



- Under SSL verification: enable SSL verification
- Select the Push Event under triggering this webhook
- **Add Webhook**



3.5 Adding Git in Jenkins



Make sure to check the Github Hook Trigger for GITScm polling while configuring the Pipeline under

Configure > Trigger > GitHub hook trigger for GITScm polling.

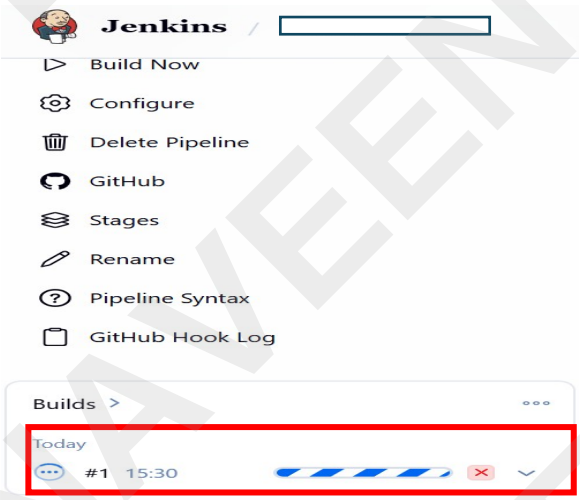
3.6 Confirming the Jenkins trigger

As soon as you commit the changes in the respective branch, the Jenkins is trigger automatically through the web hook and starts Running the pipeline.

To confirm that Jenkins jobs were triggered through a webhook, the Jenkins job configuration and build history were examined. During validation, the Jenkins **Build History** and **Console Output** were checked for indicators of webhook-based execution. The build logs showed that the job was initiated by a remote trigger rather than a manual or scheduled start.

The webhook endpoint configured in Jenkins was also verified to ensure it matched the URL registered in the source repository settings. Successful HTTP delivery responses from the repository webhook logs further confirmed that Jenkins received and processed the trigger request correctly.

Based on these checks—job trigger configuration, build cause logs, webhook delivery status, and timestamp correlation—it was concluded that the Jenkins pipeline was successfully triggered via webhook.



#1 (21 Feb 2026, 15:30:43)

Progress:

Add description

Keep this build forever

Started by GitHub push by

Started 8 min 35 sec ago
Build has been executing for 8 min 35 sec

This run spent 9.9 sec waiting in the queue.

git

Revision: c34f6c222c68d4938837e63a39b4fe93bccc7e33
Repository: https://github.com/Enkefalos-Technologies/InsurancGPT-Suite.git

- refs/remotes/origin/release_demo

No changes.

NAVEEN